

Frontend - Tag 14

Destructuring in modernem JavaScript

Lernziel: Werte aus Objekten und Arrays kurz und lesbar in eigene Variablen übernehmen - inklusive Default Values, Umbenennung und Funktionsparametern.

1. Object Destructuring

Statt jede Property einzeln über das Objekt abzurufen, kannst du mehrere Werte in einer Zeile herausziehen. Die Variablennamen müssen dabei zu den Property-Namen passen.

```
const developer = {  
  name: "Marcel",  
  role: "Frontend-Entwickler",  
  experience: 0,  
  islearning: true  
};  
  
const { name, role, isLearning } = developer;  
  
console.log(name); // Marcel  
console.log(role); // Frontend-Entwickler  
console.log(isLearning); // true
```

Merke: Nach dem Destructuring verwendest du `name` statt `developer.name`.

2. Array Destructuring

Bei Arrays zählt nicht der Name, sondern die Position. Die erste Variable erhält den ersten Array-Eintrag, die zweite den zweiten usw.

```
const skills = ["HTML", "CSS", "JavaScript"];  
const [firstSkill, secondSkill, thirdSkill] = skills;  
  
console.log(firstSkill); // HTML  
console.log(secondSkill); // CSS  
console.log(thirdSkill); // JavaScript
```

3. Werte überspringen

Ein leeres Feld zwischen zwei Kommas überspringt den Wert an dieser Position.

```
const testSkills = ["HTML", "CSS", "JavaScript", "React"];  
const [firstSkill, , thirdSkill, fourthSkill] = testSkills;  
  
console.log(firstSkill); // HTML  
console.log(thirdSkill); // JavaScript  
console.log(fourthSkill); // React
```

4. Default Values

Ein Default Value ist ein Ersatzwert. Er greift, wenn eine Property oder ein Array-Eintrag fehlt oder den Wert `undefined` hat.

```
const profile = {
  name: "Marcel",
  goal: "Frontend-Entwickler"
};

const {
  name,
  goal,
  experience = "Keine Berufserfahrung"
} = profile;

console.log(experience); // Keine Berufserfahrung
```

Wichtig: Bei `null` greift der Default Value nicht, denn `null` ist ein bewusst gesetzter eigener Wert.

5. Properties umbenennen

Mit einem Doppelpunkt legst du beim Herausziehen einen neuen Variablennamen fest. Das ist hilfreich bei Namenskonflikten oder für verständlichere Namen.

```
const profile = {
  name: "Marcel",
  goal: "Frontend-Entwickler",
  age: 29
};

const {
  name: profileName,
  goal: careerGoal,
  age
} = profile;

console.log(profileName); // Marcel
console.log(careerGoal); // Frontend-Entwickler
console.log(age); // 29
```

6. Destructuring in Funktionsparametern

Eine Funktion kann die benötigten Properties direkt im Parameter herausziehen. Das spart Zugriffe wie `profile.name` und begegnet dir später oft bei React-Props.

```
const profile = {
  name: "Marcel",
  goal: "Frontend-Entwickler",
  hasGitHub: true
};

function showProfile({ name, goal, hasGitHub }) {
  console.log(name);
  console.log(goal);
  console.log(hasGitHub);
}

showProfile(profile);
```

7. Kombination mit Template Literals

Die Abschlussübung verbindet Object Destructuring, einen Default Value, Destructuring im Funktionsparameter und Template Literals.

```
const profile = {
  name: "Marcel",
  goal: "Frontend-Entwickler"
};

function showProfile({
  name,
  goal,
  experience = "Keine Berufserfahrung"
}) {
  console.log(`Name: ${name}`);
  console.log(`Ziel: ${goal}`);
  console.log(`Erfahrung: ${experience}`);
}

showProfile(profile);
```

Ausgabe

```
Name: Marcel
Ziel: Frontend-Entwickler
Erfahrung: Keine Berufserfahrung
```

Kurzvergleich

Technik	Worauf kommt es an?
Object Destructuring	Property-Namen zählen
Array Destructuring	Position und Reihenfolge zählen
Default Value	Greift bei fehlendem Wert oder undefined
Umbenennen	property: neuerVariablenname
Funktionsparameter	Benötigte Werte direkt beim Aufruf herausziehen

Tag-14-Merksatz: Objekte werden nach Namen destructured, Arrays nach Position. Default Values machen fehlende Werte sicher, und Funktionsparameter halten den Code kompakt.